

HIP-012 — Master Implementation Plan (manual operacional da construção da SINTERA)

Objetivo: ser o **guia operacional oficial** da implementação da plataforma — completo o bastante para orientar Produto, Arquitetura, UX e Engenharia com o **mínimo de novas decisões estruturais**. **Escopo:** preparação de ambiente → app móvel (produto principal) → backend API-first → camada observacional → capacidades nativas → integrações → qualidade/publicação. Não contém código; contém arquitetura, sequência e critérios. **Status:** Approved · **Architectural Baseline** · **Versão:** 1.1 · **Histórico:** v1.0 (2026-07-20) criação; v1.1 (2026-07-20) §4 estratégia evolutiva do monorepo + fronteiras; §15b Princípios de Evolução Arquitetural. **Fonte única de roadmap:** IMPLEMENTATION_ROADMAP (este documento **não** cria planejamento paralelo — referencia-o). **Consistência:** deriva de ADR-000 · ARCH-002 · HIP-007 · HIP-008 · HIP-009 · HIP-010 · HIP-011.

1. Visão Geral

- **Objetivos:** construir a SINTERA como **plataforma mobile-first** de história de saúde, com aquisição observacional universal, backend API-first e web complementar.
- **Princípios arquiteturais** (ver ADR-000 · ARCH-002): **Mobile-First** (app = produto principal, web complementar; toda função nasce no mobile) · **API-First** (nada existe só na web; toda capacidade = API; backend desacoplado da interface) · **Arquitetura Observacional** (HIP-007: toda medida objetiva = **Observação**) · **SSOT bruto imutável + idempotente** · **Rastreabilidade** ponta a ponta · **Compartilhamento web↔mobile** (monorepo).
- **Relação com HIPs anteriores:** HIP-006 roadmap · HIP-007 observacional · HIP-008 stack app · HIP-009 sincronização · HIP-010 plano executivo · HIP-011 produto mobile · ARCH-002 mobile/API-first. Este documento os **operacionaliza**.

2. Roadmap Executivo

Toda a evolução prevista está no IMPLEMENTATION_ROADMAP (**fonte única**). Resumo das ondas de valor (HIP-010): **O1 Fundação** → **O2 Experiência principal** → **O3 Aquisição observacional** (capacidades nativas) → **O4 Integrações**. Cada onda = **produto utilizável** + aprendizado + revisão de aderência.

3. Preparação do Ambiente (checklist)

- **Contas:** Apple Developer Program (US\$99/ano) · Google Play Console (US\$25) · **Expo/EAS** (build/OTA/submit) · organização no repositório (monorepo) · acesso Supabase (já existe).

- **Certificados/assinaturas:** Apple (App ID, provisioning, push key APNs) · Android (keystore, Play signing, FCM).
- **Ambientes:** `dev` · `preview/homolog` · `prod` — variáveis por ambiente (Supabase URL/keys, Withings/env de conectores, endpoints). Segredos **nunca** no cliente; server-only.
- **Pipelines:** EAS Build (iOS/Android) · EAS Update (OTA) · EAS Submit (lojas) · CI (lint/tsc/testes) · web (Vercel).
- **Permissões (declaração):** HealthKit (iOS entitlement + NSHealth*UsageDescription) · Health Connect (Android permissions + política de privacidade) · câmera/armazenamento/notificações.

4. Estrutura do Monorepo (estratégia EVOLUTIVA — baixo risco à web em produção)

Como a web já está **em produção**, **não** fazemos migração estrutural grande agora. Evoluímos em 4 etapas (decisão da fundadora — ADR-007):

- **Etapa A (agora):** criar **workspaces** (npm) + `packages/*`, mantendo a web **exatamente onde está** (`app/`, `components/`, `lib/` na raiz). **Nada quebra.**
- **Etapa B:** criar `apps/mobile` (RN+Expo) já **consumindo** `packages/*`.
- **Etapa C:** **extrair progressivamente** para `packages/` o código compartilhável (ex.: `lib/auth.ts` → `packages/core`); web e mobile passam a usar a mesma fonte.
- **Etapa D:** quando ~70–80% do compartilhado estiver estabilizado, **mover a web para** `apps/web` — migração pequena, segura e previsível.

Estrutura-alvo:

```
sintera-platform/      (raiz = web hoje; vira workspace root)
  app/ components/ lib/ ... (web — permanece até a Etapa D → apps/web)
  apps/mobile/          (RN+Expo — Etapa B)
  packages/
    core/               domínio SINTERA: entidades, casos de uso, regras de negócio
    api-client/         comunicação HTTP (consumo das APIs)
    types/              contratos (tipos compartilhados)
    validation/         schemas Zod
    design-system/      tokens + componentes/lógica de UI compartilháveis
    config/             constantes e configuração
    utils/              APENAS utilidades genéricas
```

Fronteiras de responsabilidade (regra permanente): `packages/core` **não** é depósito de utilitários — tem responsabilidade clara (domínio). Cada pacote tem uma fronteira única; utilidades genéricas ficam em `utils`, não em `core`. Evita o "core sem fronteiras" (erro clássico de monorepo).

- **Compartilhamento:** domínio/validações/regras/contratos/cliente em `packages/*`; apps só orquestram UI.
- **Versionamento:** npm workspaces + versionamento interno; **um só contrato** web↔mobile.

- **Convenções:** TypeScript estrito; nomes de domínio da Sidebar (sidebar_ssot_taxonomia).
- **Nota design-system:** RN e web têm primitivas de UI distintas → compartilham **tokens + lógica**; camadas de componente podem ter implementação por plataforma.

5. Arquitetura do Aplicativo (React Native)

- **Organização:** por **feature module** (health-sync, timeline, exames, agenda, perfil...) sobre camadas: **auth/sessão** · **dados/offline** (cache + fila de envio) · **API** (via @sintera/core /api-client) · **UI/DS**.
- **Navegação:** bottom tabs espelhando a Sidebar (HIP-011).
- **Estado:** estado de servidor via camada de dados (cache/queries) + estado local mínimo; sem regra de negócio na UI.
- **Offline:** offline-first de leitura; captura enfileirada; sync idempotente (HIP-009).
- **Segurança:** SecureStore/Keychain/Keystore; tokens nunca em log; sessão Supabase.

6. Design System (fundação — HIP-010 O1)

Tokens (cor Van Gogh/Almond Blossom, tipografia, espaçamento) · componentes base · **princípios de navegação** · **acessibilidade** (tema_g_acessibilidade) · **animações** · **feedbacks** · **estados vazios/carregamento/erro** · boas práticas. É pré-condição das funcionalidades (consistência por toda a evolução).

7. Ordem de Implementação (sequência detalhada)

Encerramento de cada etapa = seu **critério de aceite** (§16) + revisão de aderência (HIP-010).

1. **Monorepo** + @sintera/core (tipos/contratos base) → *fecha quando web e app importam o mesmo core.*
2. **App RN+Expo** + **navegação** + **DS fundação** → *fecha quando navega com DS aplicado.*
3. **Autenticação** (app + API) → *fecha com login/logout/sessão/deep link.*
4. **Cliente de API compartilhado** → *fecha consumindo endpoint real tipado.* **[fim O1]**
5. **APIs + telas: Timeline** → **Exames/Documentos** → **Perfil** → **Agenda** (cada uma API-first) **[O2]**.
6. **Camada observacional:** endpoint /ingest + modelo Observação + projeções/NOV-001 **[O3 backend]**.
7. **Capacidades nativas:** Health Connect → Apple HealthKit → sincronização (push/offline/idempotência) **[O3 app]**.
8. **Integrações externas:** Withings (pronto) → agregador/atividade/dispositivos médicos **[O4]**.

8. Estratégia de Backend (API-first)

- **Contratos** versionados em @sintera/core (mesma fonte p/ app e web); **sem endpoint só-web**.
- **Autenticação:** Supabase Auth + RLS; service-role só server-side.

- **Uploads** (documentos/exames): endpoint de upload + storage + vínculo ao domínio; usado por app e web.
- **Sincronização:** endpoint `/ingest` (lotes de Observações, idempotente/versionado — HIP-009).
- **Versionamento de API:** compatibilidade retroativa; mudanças por adição (modelo aberto).

9. Estratégia Observacional (arquitetura + sequência, sem código)

Implementar a camada de HIP-007: **Observação** (metadados: domínio, métrica, valor/unidade, tempo/intervalo, origem, dispositivo, confiabilidade, qualidade, versão, proveniência) → **SSOT bruto** → **projeções** (Monitoramento/Composição/ Timeline) + **NOV-001** → **Indicadores** (camada derivada, V3). Reconciliação na projeção; rastreabilidade ponta a ponta. Sequência: contrato Observação → ingestão idempotente → projeções → NOV-001 → (depois) indicadores.

10. Estratégia de Capacidades Nativas

Tratadas como **capacidades nativas do app** (não conectores — ARCH-002): **Apple Health** · **Health Connect** (permissões, leitura, push → `/ingest`) · **notificações** (push APNs/FCM + NOTIF-001) · **armazenamento seguro** · **câmera/scanner** (captura de documentos) · **OCR** · **gestão de permissões**. Todas alimentam a mesma arquitetura (Observação / captura).

11. Estratégia de Integrações Futuras

Camada de conectores externos (ARCH-004): **agregadores** (Terra/Rook, por gatilho) · **fabricantes** (Withings pronto; Oura/WHOOP conforme device) · **dispositivos médicos** (CGM/MAPA/Holter) · **parceiros/FHIR/RNDS**. Todos → **Observação**; novo conector = novo adaptador, zero mudança estrutural (HIP-001).

12. Estratégia de Qualidade

- **Testes:** unitários (core/regras), de contrato (API), de componente (DS/telas), E2E por fluxo; convenção RI/ARCH/ FUNC/INT/E2E (suite_testes_capture_hub).
- **Homologação:** por onda; aceite = **comportamento observado** (governanca_aprovacao_acao_destrutiva).
- **Distribuição/métricas/observabilidade:** §13 + métricas por onda (HIP-010) + telemetria (`usage_events`), logs, crash reporting; gestão de releases via EAS.

13. Plano de Publicação (escada)

Desenvolvimento (dev build) → **TestFlight (iOS) + Play Internal (Android)** → **beta fechado** (grupo de homologação) → **beta público** (quando fizer sentido) → **produção** (App Store / Google Play). Web publica

em paralelo (Vercel). A distribuição começa na Onda 1.

14. Gestão do Projeto

- **Marcos/checkpoints:** um marco por onda + checkpoints por etapa (§7).
- **Gates entre ondas:** validação de produto → aprovação explícita da fundadora → Gate de Conformidade → revisão de aderência arquitetural → só então a próxima onda.
- **Freeze:** ao encerrar uma onda homologada (muda só por evidência).
- **Versionamento:** semântico por app/pacote; releases por onda.

15. Riscos

- **Técnicos:** limites de background sync (SO); revisão HealthKit nas lojas; divergência web↔app (mitigada por monorepo).
- **Produto:** app inflar além do MVP por onda; adoção/retenção incertas → guiar por métricas.
- **Operacionais:** contas/certificados; sign-off jurídico (Strava); custo/dependência de agregador.
- **Mitigação:** preparar contas/permisões cedo; MVP enxuto por onda; gates e revisões de aderência; contratos únicos.

15b. Princípios de Evolução Arquitetural (guia de longo prazo)

Regras para manter a arquitetura consistente à medida que equipe e base de código crescem:

- **Extensibilidade > otimização prematura** — projetar para acomodar o novo, não para otimizar cedo demais.
- **Evitar acoplamento entre domínios** — comunicação por contratos/serviços compartilhados (ADR-009).
- **Compatibilidade retroativa nas APIs** sempre que possível — evoluir por adição (modelo aberto).
- **Toda decisão estrutural relevante gera um ADR** (`docs/adr/`) — registra o *porquê*.
- **Nenhuma camada depende da interface gráfica** — domínio, sincronização e infraestrutura são UI-independent.
- **Todo código compartilhado tem PROPRIETÁRIO funcional** — nada de pacotes/módulos "sem dono"; fronteiras claras (ADR-007: `core` =domínio, `types` =contratos, etc.).

16. Critérios de Conclusão

- **Etapa concluída:** critério da etapa (§7) atendido + verde (lint/tsc/testes) + observado em build.
- **Onda concluída:** entregável utilizável + responde "o que o usuário passa a conseguir fazer?" + métricas coletando + aprovação explícita + Gate de Conformidade + revisão de aderência.
- **MVP concluído:** auth + Apple Health/Health Connect + sincronização + Timeline mínima, observado com dispositivo real.

- **Produto pronto para publicação:** ondas essenciais homologadas + qualidade/observabilidade + aprovação final.